Deploy — Controle 50K (prensa-stroke)

Guia de deploy do sistema no servidor (Proxmox) usando **deploy por código-fonte**: o servidor clona o repositório do GitHub e builda a imagem Docker localmente.

Visão geral

Item	Valor
Servidor	`root@172.16.190.143` (SSH)
Repositório	`https://github.com/IT-YorozuBR/toolTrack`
Pasta do deploy	`~/toolTrack`
Pasta antiga (fallback)	`~/prensa-stroke` (deploy por `.tar`, mantida como backup)
Container do app	`prensa-stroke`
Container do banco	`prensa-stroke-db` (serviço `db`)
Volume dos dados	`prensa-stroke_pgdata`
Rede	`prensa-stroke_default`
Project name (Compose)	`prensa-stroke` (fixado no `.env`)

> **Por que `COMPOSE_PROJECT_NAME=prensa-stroke`?** O Compose prefixa o nome do volume com o nome do projeto (por padrão, o nome da pasta). Como a pasta agora é `toolTrack`, sem essa variável o Compose criaria um volume `toolTrack_pgdata` **novo e vazio**. Fixando o project name em `prensa-stroke`, ele reaproveita o volume `prensa-stroke_pgdata` com os dados reais.

Deploy do dia a dia (fluxo recorrente)

Depois da configuração inicial já feita, todo deploy é isto:

```
cd ~/toolTrack

# 1. pega o código novo do GitHub
git pull

# 2. builda a imagem nova SEM derrubar o app antigo (fica no ar durante o build)
docker compose build

# 3. sobe a imagem nova (recria só o que mudou; downtime de segundos)
docker compose up -d

# 4. aplica migrations pendentes (ver seção "Migrations" abaixo)

# 5. confere
docker compose ps
docker logs --tail=50 prensa-stroke
```

> Se **não houver migration nova** naquele deploy, o passo 4 é dispensável. Na dúvida, rodar o passo 4 é sempre seguro (é idempotente).

Migrations

O container do app **NÃO roda migrations sozinho** no start — é preciso aplicá-las manualmente quando um deploy inclui migration nova (arquivos em `prisma/migrations/`).

Opção A — `prisma migrate deploy` (recomendada, serve para qualquer migration)

Aplica **todas** as migrations pendentes e registra no controle do Prisma:

```
cd ~/toolTrack
docker build --target builder -t prensa-migrate .
docker run --rm --network prensa-stroke_default --env-file .env \
  prensa-migrate npx prisma migrate deploy
```

Opção B — SQL manual (fallback, para uma coluna específica)

Útil se a Opção A falhar. Lê usuário/banco do `.env` automaticamente:

```
cd ~/toolTrack
PGUSER=$(grep -E '^POSTGRES_USER=' .env | cut -d= -f2)
PGDB=$(grep -E '^POSTGRES_DB=' .env | cut -d= -f2)
docker compose exec -T db psql -U "$PGUSER" -d "$PGDB" \
  -c 'ALTER TABLE "Tool" ADD COLUMN IF NOT EXISTS "ignoreManualReadings" BOOLEAN NOT NULL DE
```

Ver logs / diagnosticar erro (ex.: 500)

O container se chama `prensa-stroke`, então funciona de qualquer pasta:

```
# últimas 100 linhas
docker logs --tail=100 prensa-stroke

# acompanhar em tempo real (recarregue a tela que quebra; Ctrl+C para sair)
docker logs -f prensa-stroke
```

Erro 500 mais comum: migration não aplicada. Se o log mostrar algo como `column "ignoreManualReadings" does not exist`, rode a seção **Migrations** acima.

Rollback (voltar para o deploy antigo)

O deploy antigo (imagem do `.tar`) fica em `~/prensa-stroke` como rede de segurança. O volume de dados é o mesmo, então **não há perda de dados** ao voltar.

```
# 1. derruba o deploy novo
cd ~/toolTrack && docker compose down

# 2. restaura a imagem antiga (o build novo sobrescreveu a tag prensa-stroke:latest)
cd ~/prensa-stroke
docker load -i prensa-stroke.tar

# 3. sobe o antigo
docker compose up -d
docker compose ps
```

> Migrations já aplicadas não atrapalham o código antigo: uma coluna nova com valor padrão é simplesmente ignorada pela versão anterior.

Backup do banco (faça antes de operações de risco)

```
cd ~/toolTrack
PGUSER=$(grep -E '^POSTGRES_USER=' .env | cut -d= -f2)
PGDB=$(grep -E '^POSTGRES_DB=' .env | cut -d= -f2)
docker compose exec -T db pg_dump -U "$PGUSER" "$PGDB" > ~/backup-$(date +%F-%H%M).sql
ls -lh ~/backup-*.sql   # confirme tamanho > 0
```

Restaurar um backup:

```
cat ~/backup-ARQUIVO.sql | docker compose exec -T db psql -U "$PGUSER" -d "$PGDB"
```

Comandos Docker essenciais

Referência rápida dos comandos mais usados no dia a dia deste servidor.

Containers

```
docker ps                       # containers rodando agora
docker ps -a                    # todos (inclui parados)
docker logs prensa-stroke       # logs do app
docker logs -f prensa-stroke    # logs em tempo real (Ctrl+C para sair)
docker logs --tail=100 prensa-stroke   # últimas 100 linhas

docker restart prensa-stroke    # reinicia o app
docker stop prensa-stroke       # para
docker start prensa-stroke      # inicia

# abrir um shell DENTRO do container do app (para inspecionar)
docker exec -it prensa-stroke sh

# rodar um comando pontual dentro do banco
docker exec -it prensa-stroke-db psql -U <user> -d <db>
```

Docker Compose (sempre rodar dentro de `~/toolTrack`)

```
docker compose ps               # status dos serviços deste projeto
docker compose up -d            # sobe/atualiza em background
docker compose up -d --build    # rebuilda a imagem e sobe
docker compose build            # só builda (sem subir)
docker compose down             # derruba os containers (MANTÉM o volume)
docker compose down -v          # ⚠️ derruba e APAGA o volume (perde o banco!) — NÃO usar
docker compose restart          # reinicia os serviços
docker compose logs -f app      # logs do serviço app
docker compose logs -f db       # logs do banco
docker compose config           # mostra o compose já resolvido (útil p/ conferir volume/rec
docker compose exec db psql -U <user> -d <db>   # entra no psql do banco
```

> Diferença: `docker compose down` **preserva** o volume `prensa-stroke_pgdata` (seus dados). O `-v` apaga o volume — **nunca** use aqui.

Imagens

```
docker images                   # lista imagens
docker rmi <imagem>             # remove uma imagem
docker build -t nome:tag .      # builda a partir do Dockerfile no diretório atual
docker load -i arquivo.tar      # importa imagem de um .tar (docker save)
docker save nome:tag -o arquivo.tar   # exporta imagem para .tar
```

Volumes e rede

```
docker volume ls                # lista volumes (o do banco é prensa-stroke_pgdata)
docker volume inspect prensa-stroke_pgdata   # detalhes/localização no disco
docker network ls               # lista redes (a deste projeto é prensa-stroke_default)
```

Limpeza (liberar espaço em disco)

```
docker system df                # quanto disco Docker está usando
docker image prune              # remove imagens soltas (dangling) — seguro
docker builder prune            # limpa cache de build antigo
docker system prune             # remove tudo que não está em uso (containers parados, redes
docker system prune -a          # ⚠️ mais agressivo: remove também imagens não usadas por c
```

> Depois de vários deploys, sobram imagens antigas ocupando disco. `docker image prune` + `docker builder prune` resolvem com segurança.

Configuração inicial (one-time — já feita, aqui para referência)

Só é necessário se for montar o deploy do zero numa máquina nova:

```
# 1. clonar o repositório
cd ~
git clone https://github.com/IT-YorozuBR/toolTrack.git toolTrack
cd toolTrack

# 2. criar o .env (NÃO vem no clone — é gitignored).
#    Copie de um deploy existente OU preencha com:
#    POSTGRES_USER, POSTGRES_PASSWORD, POSTGRES_DB e
#    DATABASE_URL=postgresql://<user>:<senha>@db:5432/<db>?sslmode=disable
cp ~/prensa-stroke/.env .    # se houver um deploy antigo

# 3. fixar o project name (reaproveita o volume de dados existente)
grep -q COMPOSE_PROJECT_NAME .env || echo "COMPOSE_PROJECT_NAME=prensa-stroke" >> .env

# 4. subir
docker compose up -d --build

# 5. aplicar migrations (seção Migrations) e conferir logs
```

Observações

- O `.env` é **gitignored** e nunca vem no `git pull` / `clone` — ele vive só no servidor.
- A imagem exportada (`prensa-stroke.tar`) e dumps (`*.sql`, `backup_neon.backup`) **não** são versionados (são pesados). Apenas `Dockerfile` e `docker-compose*.yml` são.
- **Melhoria futura possível:** automatizar as migrations adicionando um serviço `migrate` no `docker-compose.yml` (roda `prisma migrate deploy` antes do app subir), eliminando o passo manual de migration a cada deploy.